Project Documentation

Date	7 July 2026
Team ID	xxxxxx
Project Name	Smart Lender
Maximum Marks	5 Marks

Smart Lender: AI-Powered Loan Eligibility Prediction Platform

Project Description

Smart Lender is a machine learning-powered web application designed to predict the creditworthiness of loan applicants, enabling banks and financial institutions to make faster, data-driven loan approval decisions. The platform trains and compares four classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost — on applicant data to evaluate the likelihood of loan repayment or default.

The application processes structured applicant inputs such as gender, marital status, education, employment status, income, loan amount, loan term, credit history, and property area. After training and evaluating all four models on an 80/20 stratified split, the best-performing model (XGBoost, at 94.7% training accuracy and 81.1% testing accuracy) is serialized as a single deployable artifact — bundled with its scaler, encoding mappings, imputation values, and feature order — and integrated into a Flask web application for real-time inference.

Beyond the core prediction, Smart Lender includes an EMI/repayment calculator, a risk-banding engine (Low/Moderate/High Risk), explainable-AI reason cards for each decision, personalized improvement suggestions for rejected applicants, and an analytics dashboard with EDA visualizations. Built with Python, Flask, and Tailwind CSS, and designed for deployment on IBM Cloud, Smart Lender gives financial analysts, credit officers, and banking professionals a fast, transparent way to reduce non-performing assets and improve the credit evaluation process.

Scenarios

Scenario 1: Fast-Track Approval for Low-Risk Applicants
A bank credit officer enters the details of a salaried applicant with a good credit history and stable income. The model predicts loan approval with high confidence, classifies the profile as Low Risk, and allows the officer to fast-track the application without manual review.

Scenario 2: High-Risk Applicant Detection
An applicant with irregular self-employment income and no credit history submits a loan request. The system flags the application as High Risk based on the trained model's output, generates explainable reason cards highlighting the weak factors, and prompts further scrutiny or document verification before proceeding.

Scenario 3: High-Volume Batch Evaluation
A financial analyst uses the platform to evaluate multiple applicants during a high-volume period. By relying on the XGBoost model's predictions and the built-in analytics dashboard, the analyst significantly reduces evaluation time while maintaining accuracy and compliance.

Technical Architecture

Description: Smart Lender utilizes a two-stage architecture — an offline training pipeline and a real-time serving application — sharing a common preprocessing layer for consistency. train_pipeline.py handles data cleaning, imputation, categorical encoding, model training, evaluation, and serialization of the winning model into best_model.pkl (containing the model, scaler, mappings, imputation values, numeric/categorical column lists, feature order, and accuracy scores). The Flask backend (app.py) loads this artifact via a lazy-singleton load_model() guard and exposes routes for the intake form, prediction, results, analytics dashboard, and EDA pages. The frontend is built with HTML, CSS, JavaScript, and Tailwind CSS using a shared design-token system across all templates.

Pipeline flow: Applicant Input → Flask Intake Form (/eligibility) → /predict Route → Preprocessing (encoding + scaling via stored mappings/scaler) → XGBoost Model Inference → Score, Risk Band, EMI Calculation, Explainable Reasons & Suggestions → /analyzing (transition) → /results → optional /dashboard and /eda for aggregate analytics.

(Note: If a database is added to the project in future, an ER diagram should be included alongside the description.)

Pre-requisites

- **Python Programming Proficiency:** Python Documentation
- **Flask Framework Knowledge:** Flask Documentation

- **Machine Learning Fundamentals:** familiarity with Scikit-learn, XGBoost, and classification metrics
- **Data Handling:** Pandas, NumPy, and Imbalanced-learn (SMOTE) for handling class imbalance
- **Data Visualization:** Matplotlib and Seaborn for EDA charts
- **HTML, CSS, JavaScript & Tailwind CSS:** Tailwind Documentation
- **Version Control with Git:** Git Documentation
- **IBM Cloud Deployment Basics:** for public hosting of the Flask application

Project Workflow

Milestone 1: Data Preprocessing & Model Selection (Sprint-1)

- **Activity 1.1:** Clean and impute missing values — fill missing categoricals with column mode and missing numerics with column median, ensuring zero nulls remain.
- **Activity 1.2:** Encode categorical features (Gender, Married, Dependents, Education, Self_Employed, Property_Area, Credit_History) into numeric codes via a defined mappings dictionary, with fallback-to-mode handling for unseen values.
- **Activity 1.3:** Train and compare Decision Tree, Random Forest, KNN, and XGBoost on an 80/20 stratified split, recording train/test accuracy for each in an accuracies dictionary.
- **Activity 1.4:** Serialize the best-performing model (XGBoost) along with its scaler, mappings, imputation values, column lists, and feature order into a single best_model.pkl artifact, loaded via a lazy-singleton load_model() function.

Milestone 2: Core Prediction Functionality (Sprint-2)

- **Activity 2.1:** Build the applicant intake form (loanEligibility.html) rendering all 11 required fields, posting to /predict.
- **Activity 2.2:** Implement the /predict inference route — reading form fields, encoding via stored mappings, assembling a single-row DataFrame in the correct feature order, and scaling numeric columns.
- **Activity 2.3:** Compute a decision-consistent eligibility score (approved results always ≥ 50, rejected always < 50) regardless of raw model probability.
- **Activity 2.4:** Classify each profile into a risk band — Low Risk (≥75), Moderate Risk (60–74), or High Risk (<60) — with matching descriptions.

Milestone 3: Explainability & Financial Insights (Sprint-3)

- **Activity 3.1:** Generate explainable AI reason cards (credit history, DTI coverage, property area, employment/education) flagged as positive or negative.
- **Activity 3.2:** Compute EMI, total interest, and total payable using the standard amortization formula at an 8% annual rate.
- **Activity 3.3:** Generate personalized improvement suggestions — positive suggestions for approved profiles, targeted suggestions addressing weak factors for rejected profiles.
- **Activity 3.4:** Build an animated transition (/analyzing) between form submission and the results page.

Milestone 4: Analytics Dashboard, EDA & Deployment (Sprint-4)

- **Activity 4.1:** Build the analytics dashboard (/dashboard) showing live record/approved/rejected counts and the model accuracy comparison, with a hardcoded fallback on data-read failure.
- **Activity 4.2:** Build the EDA page (/eda) rendering pre-generated univariate, bivariate, and multivariate chart images.
- **Activity 4.3:** Apply a consistent Tailwind CSS design system (colors, spacing, type scale) across all templates.
- **Activity 4.4:** Validate the end-to-end user journey (form → predict → analyzing → results → dashboard → eda) across varied test profiles, then deploy the application on IBM Cloud.

Milestone 5: Conclusion

Conclusion

Smart Lender is a machine learning platform built with Python and Flask that streamlines loan eligibility assessment for financial institutions. By comparing four classification models and deploying the best-performing one (XGBoost) behind a clean, Tailwind-styled interface, the project delivers real-time, explainable, and financially grounded lending decisions — complete with EMI calculations, risk banding, and personalized recommendations. The project's modular design, with a shared preprocessing layer between training and serving, ensures maintainability and reusability. Looking ahead, Smart Lender offers opportunities for expansion such as database-backed applicant history, model retraining pipelines, and role-based access for credit officers versus analysts, positioning it to grow from a prediction tool into a comprehensive credit-risk management platform.